Evaluating Verifiability in the Privacy Preserving Data Analytics Engine (E-Privacy)

Dilmith D. L. S., Hansana O. G. V., Vidyananda H. K. H.

September 25, 2026

1 Introduction

In E-Privacy, a Data Owner (e.g. a bank) encrypts its records locally with Paillier (2048-bit modulus), uploads only ciphertext to the cloud, and asks the cloud's Homomorphic Compute Engine to return aggregates such as the sum and average of `Transaction_Amount` over its 2,000,000 transaction records. Confidentiality is guaranteed because the cloud never holds the private key. *Correctness*, however, is not: an untrusted cloud could silently drop records, include stale ones, or return a wrong ciphertext, and the Data Owner would decrypt a plausible-looking but wrong number.

The Swiss e-voting system faces the same problem with ballots and tallies, and answers it with two properties, which this report applies to E-Privacy:

- **Individual verifiability** – the Data Owner can check that *its own* uploaded records were stored as uploaded and were actually used in the query it asked for.
- **Universal verifiability** – *any* third party (e.g. a financial regulator or external auditor) can check that the published result was correctly computed from the committed ciphertexts and correctly decrypted, without the private key and without seeing any individual record.

1.1 Assumptions

V1. As in the SRS (A2, A3), there is one trusted Data Owner per deployment holding the Paillier private key, and the cloud is honest-but-curious; verifiability is the mechanism that turns any deviation by the cloud into a *detectable* event.

V2. The Verifiable Computation & Audit Module (FR-6) is extended into an append-only, hash-chained **bulletin board**, readable by the Data Owner and by auditors authorised through the Access Control & Policy Engine (FR-4).

V3. Each board entry is hashed with SHA-256 and signed by the cloud with ECDSA-P256, so the cloud cannot later deny or rewrite what it published.

V4. Only aggregates over at least a policy-defined minimum number of records are published (FR-4), so revealing an aggregate result does not reveal an individual value.

1.2 Mapping from E-Voting to E-Privacy

Property	E-voting (Swiss model)	E-Privacy
Cast as intended	Voter checks return codes for their choices	Data Owner's client encrypts locally (FR-2) and spot-checks its own ciphertexts before upload
Recorded as cast	Vote is stored unchanged on the server	Each uploaded ciphertext is committed as a Merkle leaf; the Data Owner gets a signed inclusion receipt
Counted as recorded	Tally uses exactly the recorded ballots	Aggregate $C = \prod c_i \bmod n^2$ is computed over exactly the committed leaves
Correct decryption	Proof that the tally was decrypted correctly	Data Owner publishes result m with Paillier randomness r; anyone checks $g^m r^n \equiv C \pmod{n^2}$

Table 1: Verifiability properties of e-voting and their E-Privacy counterparts.

2 Protocol

Figure 1 shows how both properties fit into the existing E-Privacy query flow (upload → API Gateway → Homomorphic Compute Engine → Audit Log → local decryption).

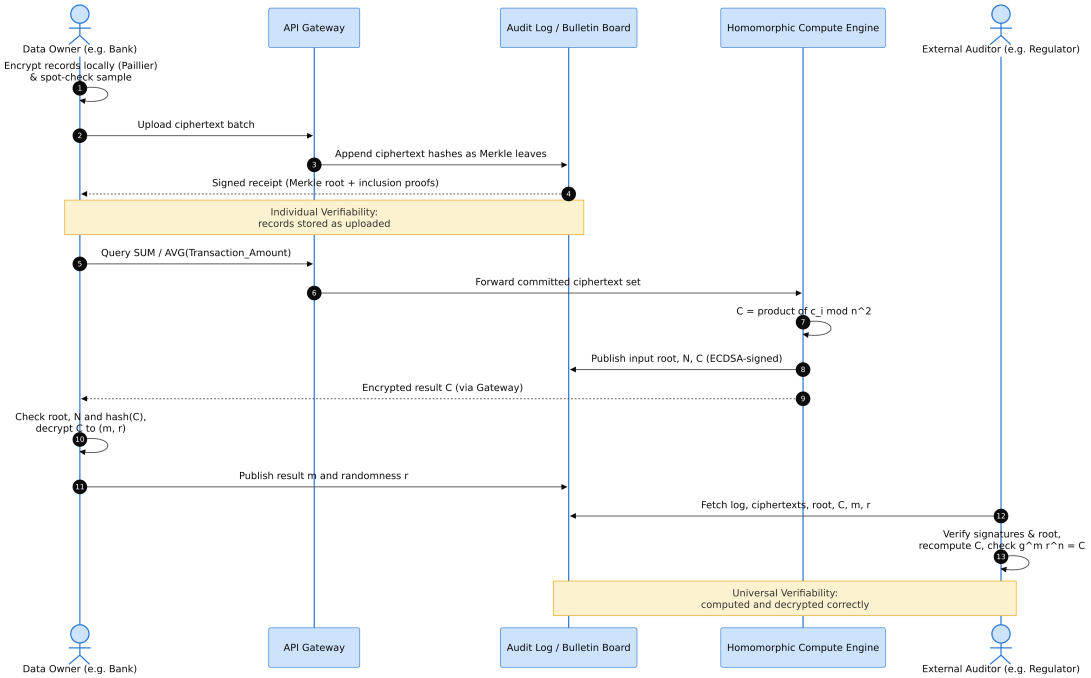


Figure 1: Individual and universal verifiability in the E-Privacy query flow.

2.1 Individual Verifiability (Data Owner)

1. **Encrypted as intended.** Encryption happens only in the Data Owner's Local Encryption Client (FR-2), never in the cloud. Before upload, the client decrypts a

random sample of the ciphertexts it just produced and compares them with the source rows, catching a faulty or tampered client library.

2. **Stored as uploaded.** When the API Gateway accepts an upload batch, the SHA-256 hash of every ciphertext c_i becomes a leaf of a Merkle tree on the bulletin board. The Gateway returns a signed receipt holding the new Merkle root and the inclusion proof for each leaf. The Data Owner keeps its own list of hashes and can check at any time that each record is still in the tree, unmodified. For the 2M-record dataset, each proof is only about 21 hashes ($\lceil \log_2 2{,}000{,}000 \rceil$).

3. **Used in my query.** For each query, e.g. `SUM(Transaction_Amount)`, the Compute Engine publishes the Merkle root and the leaf count N of the exact input set it aggregated. The Data Owner checks that the root equals the one in its latest receipt and that $N = 2{,}000{,}000$. A dropped, extra or substituted record changes the root, so the result is rejected before it is trusted.

4. **Result as computed.** The encrypted result returned by the Gateway must hash to the value the Compute Engine signed on the board; otherwise the result was swapped in transit or by the Gateway.

2.2 Universal Verifiability (Any Auditor)

An auditor who holds only public information – the Paillier public key (n, g) and the bulletin board – performs four checks:

1. **Log integrity:** recompute the SHA-256 hash chain and verify every ECDSA-P256 signature, so no entry was edited or removed after publication.

2. **Input set:** rebuild the Merkle root from the published ciphertexts and confirm it matches the root the Compute Engine signed for the query.

3. **Computation:** recompute $C' = \prod_{i=1}^{N} c_i \bmod n^2$ and confirm $C' = C$, the signed encrypted sum. Paillier addition is deterministic on public ciphertexts, so this needs no key. The average is then m/N with N public, so it needs no separate proof.

4. **Decryption:** Paillier decryption also recovers the randomness r of a ciphertext. The Data Owner publishes the sum m together with r, and the auditor checks $g^m \cdot r^n \equiv C \pmod{n^2}$. A false m cannot pass this check, so even the Data Owner cannot misreport the result to a regulator.

Checks 1–3 cost about the same as the original aggregation. Per the Computation Overhead benchmark, that is roughly 5 minutes of modular multiplication for 2M records at 2048 bits, over about 1.01 GB of ciphertext, so independent auditing is practical.

3 What Verifiability Does and Does Not Guarantee

- **Detection, not prevention.** A dishonest cloud can still drop records or return a wrong ciphertext, but it cannot do so *silently*: the Merkle root, the ciphertext product or a signature will not match. A cloud that refuses to publish at all causes a visible availability failure, not a hidden correctness failure.

- **No loss of confidentiality.** The board contains only ciphertexts, hashes, signatures and the final aggregate. Individual ciphertexts reveal nothing without the private key, and the published r belongs to the aggregate ciphertext only, so it reveals nothing about any single record.
- **Scope.** Checks 3 and 4 cover the additive queries E-Privacy supports (sum, count, average). Queries outside that scope (A1 in the SRS) would need extra proofs, such as zero-knowledge proofs, which are out of scope for this design.

4 Conclusion

E-Privacy achieves individual verifiability through local encryption with spot-checks, signed Merkle inclusion receipts, and a per-query input-set root that the Data Owner compares against its own records. It achieves universal verifiability because Paillier aggregation can be recomputed by anyone from public ciphertexts, and Paillier decryption can be proven by revealing the aggregate's randomness. Together these let a Data Owner and an external auditor both confirm that a result such as the total of 2,000,000 transaction amounts is complete and correct, without trusting the cloud and without anyone other than the Data Owner seeing a single plaintext record.